

Descripción del entorno (Cliff Walking)

Es un grid de 4 filas × 12 columnas (48 estados). El agente empieza en la esquina inferior izquierda (3, 0). El objetivo está en la esquina inferior derecha (3, 11). Hay un "acantilado" (cliff) en la fila inferior entre las columnas 1 y 10. Si caes en el cliff recibes -100 y regresas al inicio. Todas las demás transiciones dan -1. Acciones: 0=↑, 1=↓, 2=←, 3=→.

```
import gymnasium as gym
import numpy as np
import matplotlib.pyplot as plt
from collections import defaultdict
import seaborn as sns
```

Gymnasium es la biblioteca estándar de Python para Reinforcement Learning. Es la continuación oficial de la biblioteca OpenAI Gym, que OpenAI dejó de mantener activamente desde 2022. El equipo que mantenía Gym decidió crear Gymnasium bajo la Farama Foundation.

Gymnasium proporciona:

Un API estándar muy simple y consistente para definir entornos de RL. Una gran colección de benchmarks listos para usar. Herramientas para crear tus propios entornos personalizados. Esto permite que investigadores y desarrolladores puedan comparar algoritmos de forma justa, ya que todos usan el mismo formato de interacción.

CONCEPTOS BASICOS

Q(s, a): Es la función de acción-valor. Representa la recompensa esperada que obtendremos si estamos en el estado s, tomamos la acción a, y luego seguimos la mejor política posible a partir de ahí. Ambos algoritmos son tabulares (usan una tabla Q). Ambos usan ϵ -greedy para explorar (tomar acciones aleatorias con probabilidad ϵ). Ambos actualizan Q usando la ecuación de Bellman.

HIPERPARAMETROS

Start coding or generate with AI.

```
ALPHA = 0.1      # tasa de aprendizaje
GAMMA = 1.0      # factor de descuento
EPSILON = 0.1    # e-greedy
NUM_EPISODES = 1000
MAX_STEPS = 200

env = gym.make('CliffWalking-v1')
```

Devuelve un objeto env que tiene:

- env.reset() → reinicia el episodio
- env.step(action) → ejecuta una acción
- env.action_space.n → número de acciones (4: arriba, abajo, izquierda, derecha)

E-GREEDY

```
def epsilon_greedy(Q, state, epsilon, n_actions):
    if np.random.random() < epsilon:
        return np.random.randint(n_actions)
    else:
        return np.argmax(Q[state])
```

Esta función decide qué acción tomar:

Con probabilidad epsilon (10%) toma una acción aleatoria. Con probabilidad 1-epsilon toma la acción con mayor valor Q.

Q-LEARNING

El agente está en estado s y elige acción a usando ϵ -greedy. Ejecuta la acción y llega al estado s' recibiendo recompensa r. Para actualizar Q, no usa la acción que realmente va a tomar después, sino que usa la mejor acción posible en el siguiente estado ($\max Q(s', a')$). Esta actualización asume que en el futuro siempre tomará la mejor acción (política greedy). Por eso se dice que es off-policy, está aprendiendo sobre una política óptima mientras sigue una política de exploración (ϵ -greedy).

Ventajas de Q-Learning:

Tiende a converger a la política óptima. Aprende el valor real del estado, incluso si durante el aprendizaje está explorando.

Desventaja en Cliff Walking:

Como siempre considera el max, aprende que ir pegado al acantilado es óptimo. Pero durante la exploración, cuando toma acciones aleatorias, cae frecuentemente al cliff lo que lleva a recompensas muy negativas.

```
def q_learning(env, num_episodes=NUM_EPISODES, alpha=ALPHA, gamma=GAMMA, epsilon=EPSILON):
    Q = defaultdict(lambda: np.zeros(env.action_space.n))
    rewards_per_episode = []

    for episode in range(num_episodes):
        state, _ = env.reset()
        total_reward = 0

        for _ in range(MAX_STEPS):
            action = epsilon_greedy(Q, state, epsilon, env.action_space.n)
            next_state, reward, terminated, truncated, _ = env.step(action)
            done = terminated or truncated

            # Actualización Q-Learning
```

```
        best_next_action = np.argmax(Q[next_state]) # encuentra la mejor accion en el siguiente estado.
        td_target = reward + gamma * Q[next_state][best_next_action] # es el "objetivo" que queremos alcanzar (recompensa + valor estimado)
        Q[state][action] += alpha * (td_target - Q[state][action]) # se calcula el error TD (td_target - Q[state][action]) y se actualiza

        total_reward += reward
        state = next_state

    if done:
        break

    rewards_per_episode.append(total_reward)

return Q, rewards_per_episode
```

SARSA

El agente está en s, elige acción a usando ϵ -greedy. Ejecuta a, llega a s', recibe r. Ahora elige la siguiente acción a' también con ϵ -greedy (la misma política que usó para elegir a). Usa el valor de esa acción a' que realmente va a tomar para actualizar. Por eso se llama on-policy: está aprendiendo el valor de la política que realmente está ejecutando (incluyendo la exploración).

```
def sarsa(env, num_episodes=NUM_EPISODES, alpha=ALPHA, gamma=GAMMA, epsilon=EPSILON):
    Q = defaultdict(lambda: np.zeros(env.action_space.n))
    rewards_per_episode = []

    for episode in range(num_episodes):
        state, _ = env.reset()
        action = epsilon_greedy(Q, state, epsilon, env.action_space.n)
        total_reward = 0

        for _ in range(MAX_STEPS):
            next_state, reward, terminated, truncated, _ = env.step(action)
            done = terminated or truncated

            # Elegir la siguiente acción con la misma política ( $\epsilon$ -greedy)
            next_action = epsilon_greedy(Q, next_state, epsilon, env.action_space.n)

            # Actualización SARSA
            td_target = reward + gamma * Q[next_state][next_action] if not done else reward
            Q[state][action] += alpha * (td_target - Q[state][action])

            total_reward += reward
            state = next_state
            action = next_action

        if done:
            break

        rewards_per_episode.append(total_reward)

    return Q, rewards_per_episode
```

Diferencias:

En SARSA elegimos la primera acción fuera del ciclo. SARSA elige la siguiente acción (next_action) usando también ϵ -greedy. Si el episodio no ha terminado, usa $Q[next_state][next_action]$. Si el episodio terminó (done = True), solo usa la recompensa r (no hay futuro). En SARSA debemos actualizar también la variable action para el siguiente paso.

ENTRENAMIENTO

```
print("Entrenando Q-Learning...")
Q_q, rewards_q = q_learning(env)

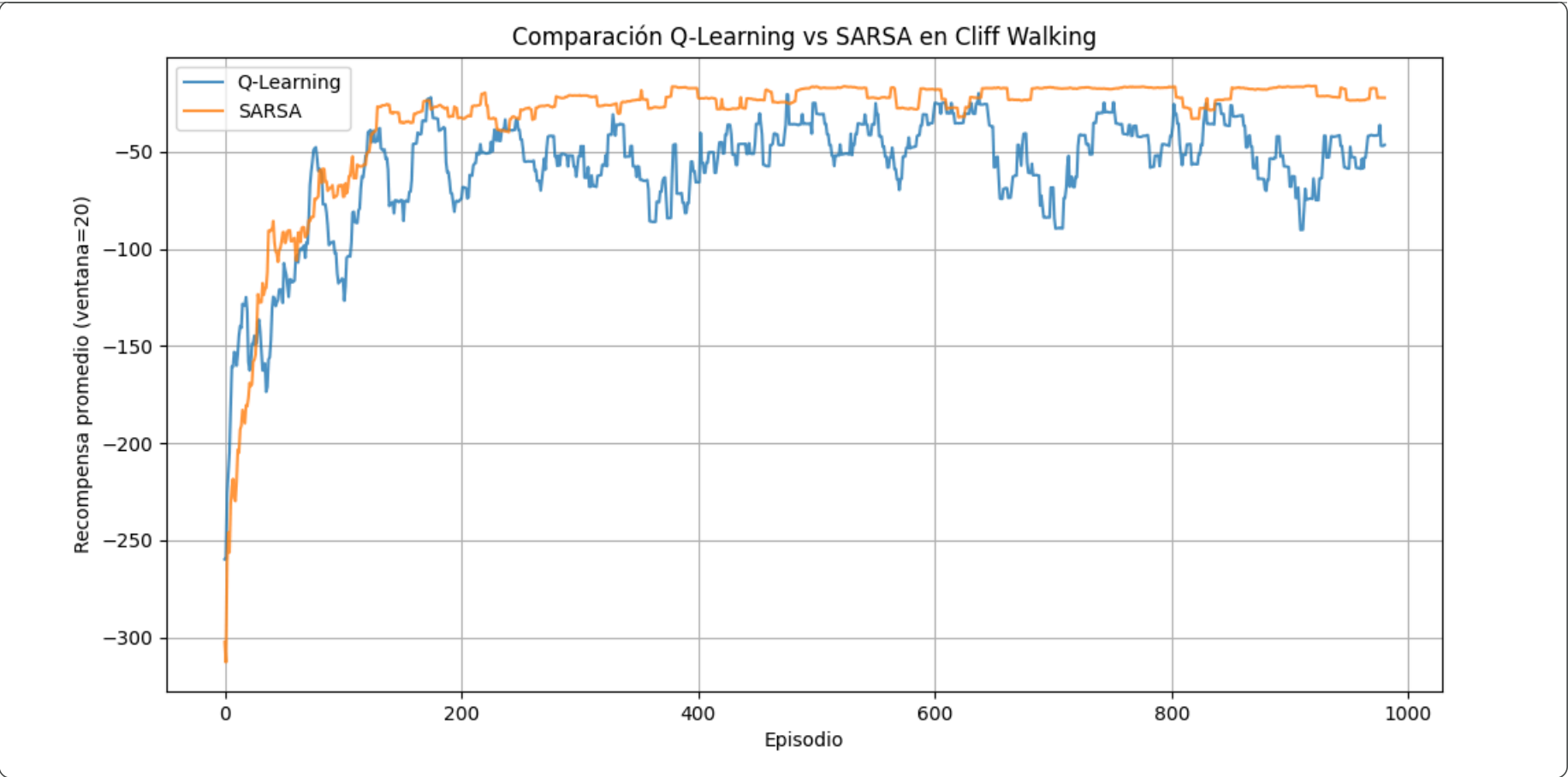
print("Entrenando SARSA...")
Q_s, rewards_s = sarsa(env)
```

Entrenando Q-Learning...
Entrenando SARSA...

GRAFICAS DE RECOMPENSA

```
def moving_average(data, window=20):
    return np.convolve(data, np.ones(window)/window, mode='valid')

plt.figure(figsize=(12, 6))
plt.plot(moving_average(rewards_q), label='Q-Learning', alpha=0.8)
plt.plot(moving_average(rewards_s), label='SARSA', alpha=0.8)
plt.title('Comparación Q-Learning vs SARSA en Cliff Walking')
plt.xlabel('Episodio')
plt.ylabel('Recompensa promedio (ventana=20)')
plt.legend()
plt.grid(True)
plt.show()
```



Para este ejercicio, SARSA ve que la próxima acción (elegida con ϵ -greedy) a veces es caer al cliff, por k lo que le asigna un valor muy malo a estar cerca del borde. Entonces prefiere alejarse un poco del acantilado para evitar el riesgo. Q-Learning ignora ese riesgo porque siempre asume “tomaré la mejor acción posible”, por lo que sigue considerando óptimo ir por el borde.

De forma general y en dependencia de la implementación, Q-Learning suele lograr mejor recompensa final, pero con más variabilidad y caídas. SARSA es más estable, pero su política final suele ser ligeramente más larga (recompensa promedio un poco peor). Q-Learning es más optimista y agresivo. SARSA es más realista y cauteloso.